Phase 3: Optimization, Scaling, and Final Evaluation for a Search Engine Retrieval Pipeline Using Python

Student Name: Gyan Bahadur Tamang

ID: 42195

E-mail: gtamang42195@ucumberlands.edu

Department of Computer Science

University of the Cumberlands

Williamsburg, KY 40769

Algorithms and Data Structures (MSCS-532-B01)

April 2, 2026

Contents

1. Optimization Techniques.....3

2. Scaling Strategy.....5

3. Advanced Testing and Validation.....6

4. Performance Analysis (Baseline vs Optimized).....6

5. Final Evaluation.....8

References.....9

1. Optimization Techniques

The Phase 2 POC implementation was successful in terms of functional correctness, but it revealed several major performance bottlenecks as data size increased. These were a vocabulary rebuild bottleneck, where every document addition/removal operation caused the rebuilding of the trie from all documents, a deletion bottleneck where every document removal operation caused scanning through all terms in the global index, inefficient Boolean AND intersection operations over unsorted sets, and limited extensibility of the ranking component to only TF-IDF scoring. These bottlenecks caused superlinear performance degradation and poor scaling with increasing data size.

The Phase 3 implementation has resolved the above-mentioned bottlenecks by optimizing the system. Incremental vocabulary updates were achieved by using the became_present and became_absent flags in the inverted index. This ensured the trie was updated and not rebuilt. Targeted document deletion was achieved by introducing a term map per document (self._doc_term_counts) to count terms per document. This allowed the deletion operation to target the terms in the deleted document. Optimization in the implementation of the Boolean AND operation was achieved by sorting the posting lists by size before intersection and introducing an early exit when the intersection result is empty. Flexibility in implementation was achieved by introducing a new BM25 ranker alongside the existing TF-IDF implementation. This was achieved by introducing an IDF cache and invalidation based on version numbers to ensure accuracy during system mutations. Data structure observability was achieved by introducing index versioning, vocabulary size tracking, average document length metrics, and document term inspection.

Significant conceptual improvements in algorithmic complexity were attained. The cost of maintaining the vocabulary changed from approximately O(N x L) in Phase 2 to approximately O(T_doc) incremental updates in Phase 3, where N is the number of documents and L is the average number of tokens per document. The cost of deleting a document changed from O(V), which scanned the entire vocabulary, to O(U_doc), where U_doc is the number of unique terms in the deleted document. The unoptimized order in AND queries was replaced with the smallest-first intersection approach to minimize the size of intermediate sets more rapidly.

The following code snippets describe the optimizations performed :

Git Repo; https://github.com/Gyanvlon/UC-Algo/tree/main/Search%20Engine/Phase3%20Delivery3

```
Snippet A: incremental vocabulary updates using mutation deltas (from
`search_engine.py`)

```python

def _apply_vocabulary_delta(self, delta: DocumentDelta) -> None:

   for term in delta.became_present:

      self.vocabulary.insert(term)

   for term in delta.became_absent:

      self.vocabulary.delete(term)

```

Snippet B: targeted deletion using per-document term maps (from `inverted_index.py`)

```python

def remove_document(self, doc_id: str) -> DocumentDelta:

   old_term_counts = self._doc_term_counts[doc_id]

   old_terms = set(old_term_counts.keys())

   for term in old_terms:

      postings = self._postings.get(term)

      if postings and doc_id in postings:

         del postings[doc_id]

         if not postings:

            del self._postings[term]

```

Snippet C: AND query optimization with smallest-first intersections (from
`inverted_index.py`)
```

```python
posting_sets.sort(key=len)
result = posting_sets[0].copy()
for postings in posting_sets[1:]:
   result.intersection_update(postings)
   if not result:
      break
return result
```

Snippet D: BM25 scoring support (from `search_engine.py`)

```python
idf = math.log(1.0 + (n_docs - df + 0.5) / (df + 0.5))
denominator = tf + k1 * (1.0 - b + b * (doc_len / avg_doc_len))
score += idf * ((tf * (k1 + 1.0)) / denominator)
```

---

2. Scaling Strategy

The search engine was extended to work with larger datasets by reducing global operations and adding localized updates. The major techniques employed were avoiding recalculating the index, restricting mutation work to specific terms, and adding alternatives to ranking. These alternatives include TF-IDF and BM25, depending on the context. Various techniques were employed to ensure memory-based design. These include adding a per-document term map to reduce deletion time at the cost of additional memory, deleting from the trie with occurrence counters (pass_count and terminal_count), and invalidating the cache through versioning to ensure no stale ranking information is provided. Challenges that arose during development were effectively addressed. These challenges include safely removing terms from the trie during repeated insertions. This was achieved by

adding pass_count and terminal_count. Another challenge was cache correctness during index mutations. This was achieved by incrementing the global version whenever a mutation occurred. Backward compatibility with existing tests and APIs was ensured by preserving the default search behavior (ranker = 'tfidf').

3. Advanced Testing and Validation

The project has an extensive set of tests, including both correctness and stress tests. For instance, tests in tests/test_data_structures.py verify correctness in index mutation and versioning, as well as trie delete correctness with non-destructive removal. In tests/test_search_engine.py, tests verify correctness for AND/OR retrieval, incremental vocabulary updates during document replacement, BM25 correctness, and ranker input validation. In tests/test_phase3_scaling.py, there is a special scaling test in which 1200 documents are indexed with a structured token distribution, correctness of retrieval is verified, correctness of retrieval in bounded top-k results for large input is verified, and batch deletion with post-deletion query robustness verification.

Stress tests and validation include large insertion waves of 1200 documents with repetitive and rotating terms, batch deletion of every third document, prefix suggestions after add/remove transitions, ranker arguments, and empty-query validation. All tests in the set of 17 tests ran successfully in 0.076 seconds, yielding the result ‘OK’. Some of the tests include ‘test_index_and_query_large_dataset’, ‘test_replace_document_updates_vocabulary_incrementally’, ‘test_bm25_ranking_is_supported’, ‘test_delete_removes_only_target_word’, among others.

4. Performance Analysis (Baseline vs Optimized)

This was done using a special benchmark script (phase3_benchmark.py), which compared the Phase 2 baseline engine with the Phase 3 optimized engine. The baseline engine performed full trie rebuilds on every mutation, global term scanning on deletes, and TF-IDF query path. The optimized engine performed incremental updates on the trie, targeted deletes, and both TF-IDF and BM25 ranking.

The performance results obtained from the above comparison are tabulated below:

dataset_size	baseline_build_s	optimized_build_s	baseline_query_ms	optimized_tfidf_ms	optimized_bm25_ms	baseline_delete_s	optimized_delete_s	build_speedup_x	query_speedup_x	delete_speedup_x
100	0.103471	0.003756	0.441	0.639	0.704	0.036436	0.000506	27.55	0.69	71.97
200	0.400052	0.006879	0.827	0.984	1.365	0.144437	0.000799	58.16	0.84	180.79
400	1.700064	0.012080	1.876	1.914	3.019	0.601160	0.001419	140.73	0.98	423.53

The benchmark results demonstrate dramatic improvements in build-time scalability, reaching up to 140.73× faster at 400 documents. Deletion performance showed the greatest gains, up to 423.53× faster, confirming the successful elimination of global postings scans. TF-IDF query latency remained comparable to the baseline in this synthetic OR-heavy workload; however, the substantial gains in the update path make the optimized engine far superior for dynamic-index scenarios. BM25 ranking incurs a higher query cost than TF-IDF, representing a deliberate trade-off favoring retrieval quality over raw speed.

Result:

```
(.venv) PS D:\Edu\UC\Algo\Search Engine\Phase3 Delivery3> uv run python main.py
AND Query: search index

OR Query: search graph
1. doc-004 (score=0.2738)
2. doc-002 (score=0.2395)

OR Query with BM25: search graph
1. doc-004 (score=1.2921)
2. doc-002 (score=1.2206)

Term suggestions for 'in': ['index', 'indexing', 'influence', 'inverted']

Removing doc-003 and re-running OR query...
1. doc-004 (score=0.2419)
2. doc-002 (score=0.2116)
```

```
(.venv) PS D:\Edu\UC\Algo\Search Engine\Phase3 Delivery3> uv run python -m unittes
t discover -s tests -p "test_*.py" -v
test_add_search_and_remove (test_data_structures.InvertedIndexTests.test_add_searc
h_and_remove) ... ok
test_index_stats_and_version_progress (test_data_structures.InvertedIndexTests.tes
t_index_stats_and_version_progress) ... ok
test_invalid_document_inputs_raise (test_data_structures.InvertedIndexTests.test_invalid_document_inputs_raise) ... ok
test_invalid_k_raises (test_data_structures.TopKHeapTests.test_invalid_k_raises) ... ok
test_maintains_highest_k_scores (test_data_structures.TopKHeapTests.test_maintains_highest_k_scores) ... ok
test_delete_removes_only_target_word (test_data_structures.TrieTests.test_delete_removes_only_target_word) ... ok
test_insert_contains_and_prefix (test_data_structures.TrieTests.test_insert_contains_and_prefix) ... ok
test_index_and_query_large_dataset (test_phase3_scaling.Phase3ScalingTests.test_index_and_query_large_dataset) ... ok
test_bm25_ranking_is_supported (test_search_engine.SearchEngineTests.test_bm25_ranking_is_supported) ... ok
test_empty_query_returns_no_results (test_search_engine.SearchEngineTests.test_empty_query_returns_no_results) ... ok
test_invalid_inputs_raise_value_error (test_search_engine.SearchEngineTests.test_invalid_inputs_raise_value_error) ... ok
test_remove_document_updates_search_and_suggestions (test_search_engine.SearchEngineTests.test_remove_document_updates_search_and_suggestions) ... ok
test_replace_document_updates_vocabulary_incrementally (test_search_engine.SearchEngineTests.test_replace_document_updates_vocabulary_incrementally) ... ok
test_search_and_mode_returns_intersection (test_search_engine.SearchEngineTests.test_search_and_mode_returns_intersection) ... ok
test_search_or_mode_returns_union (test_search_engine.SearchEngineTests.test_search_or_mode_returns_union) ... ok
test_suggestions_are_prefix_filtered (test_search_engine.SearchEngineTests.test_suggestions_are_prefix_filtered) ... ok
test_top_k_limits_results (test_search_engine.SearchEngineTests.test_top_k_limits_results) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.079s

OK
```

5. Final Evaluation

The optimized search engine has several key strengths, including update path scalability for mutable datasets, a better architecture for production-like continuous indexing workloads, modular rankers for quality experimentation, and thorough testing with stress and mutation tests.

The current limitations of the optimized search engine include its benchmark, which is based on CPU time and provides only qualitative memory analysis; synthetic benchmark data that does not perfectly mimic natural language; and static BM25 parameters that are not domain-specific.

The opportunities for future improvements to the optimized search engine include memory profiling with tracemalloc, reporting memory peaks, phrase and proximity indexing with positional postings, stopwords, stemming/lemmatization, persistent on-disk index segments with merge, and hybrid ranking models with doc priors and field weights.

References

1. Büttcher, S., Clarke, C. L. A., & Cormack, G. V. (2016). Information Retrieval: Implementing and Evaluating Search Engines. MIT Press.

2. Croft, W. B., Metzler, D., & Strohman, T. (2010). Search Engines: Information Retrieval in Practice. Addison-Wesley.

3. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389. https://doi.org/10.1561/1500000019

4. Zobel, J., & Moffat, A. (2006). Inverted files for text search engines. ACM Computing Surveys, 38(2), 6-es. https://doi.org/10.1145/1132956.1132959